PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

API REST CON EXPRESS, MYSQL

Presentado a:	Alexandra Guevara
Por Aprendiz:	**Luis fernando conejo**
Ficha:	3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

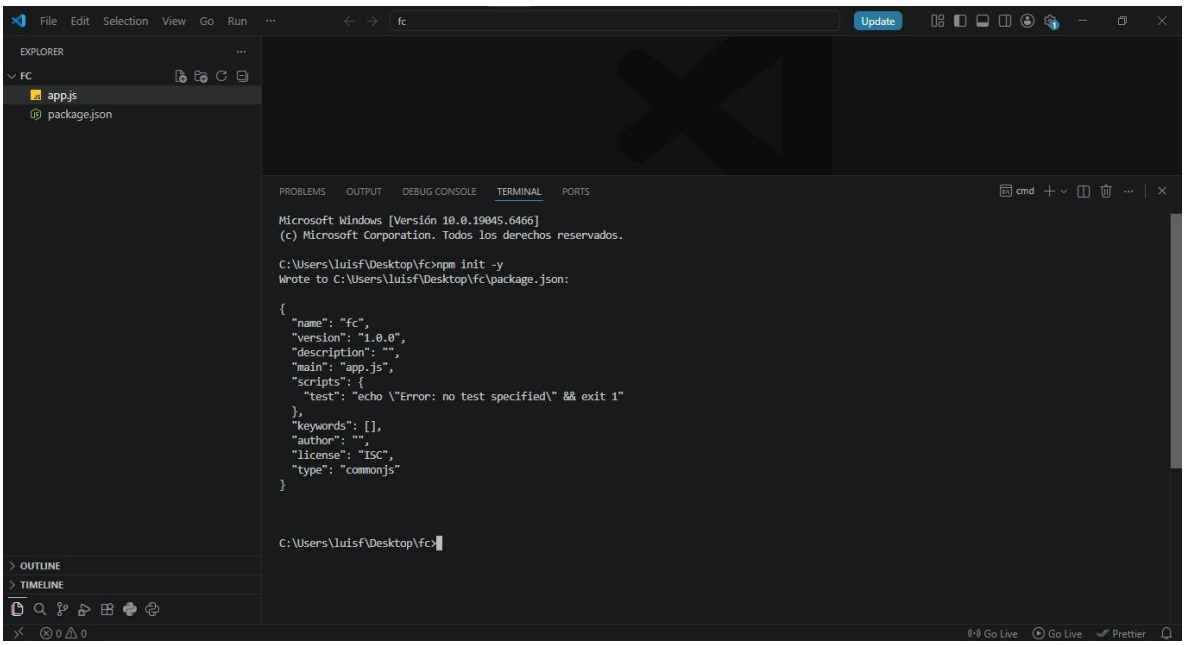
Popayán, día 26 del 10 del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Explicación de comandos y configuración del proyecto

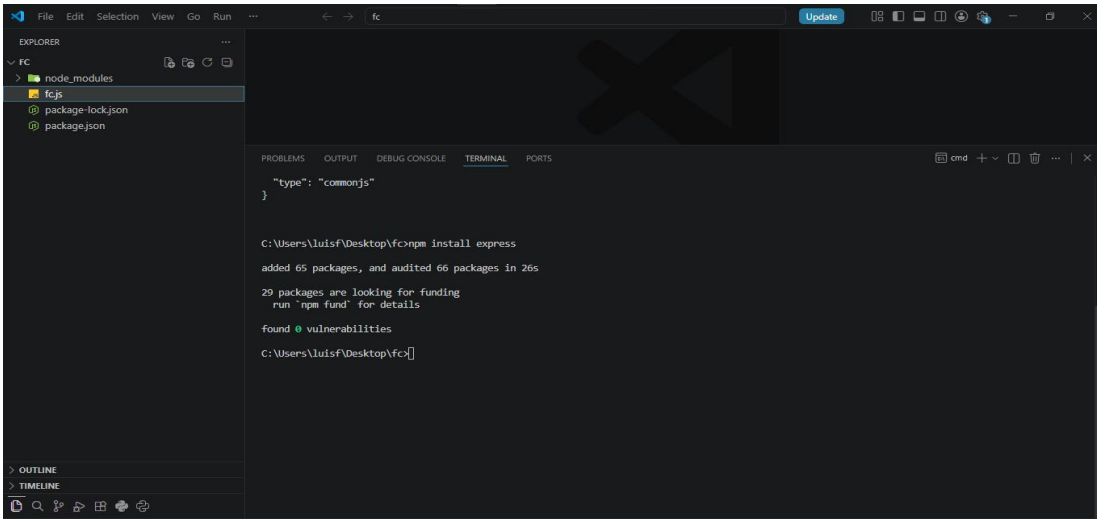
npm init -y

Inicializa el proyecto de Node.js creando el archivo package.json de golpe



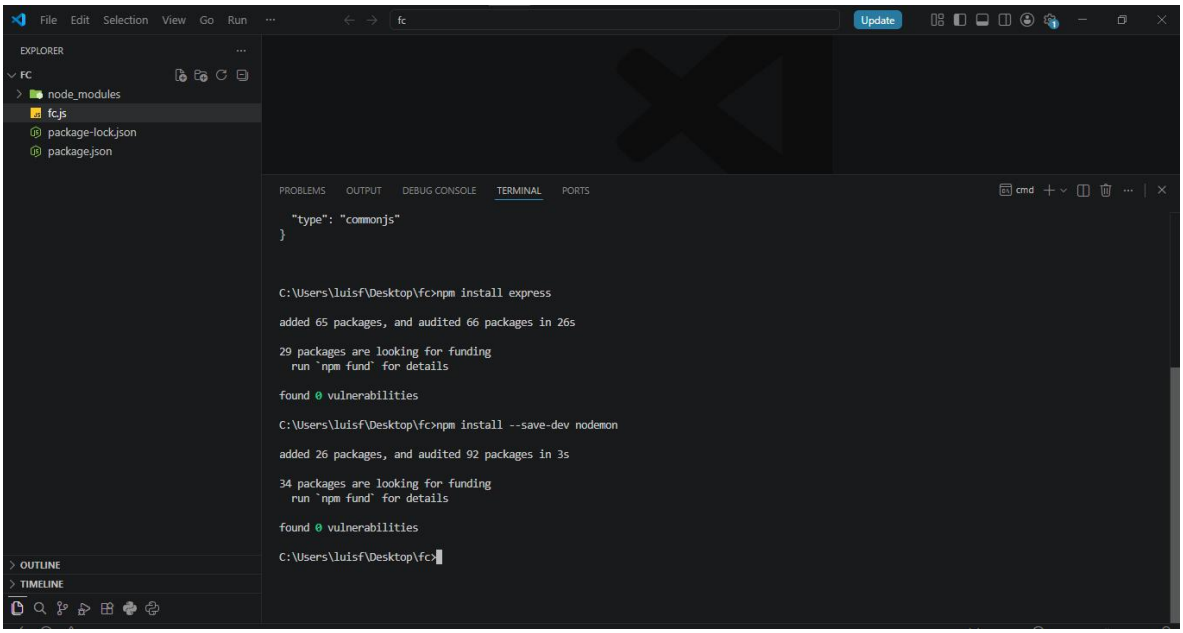
npm install express

s el núcleo de nuestro servidor web. Nos arma la estructura básica y nos deja crear las rutas (/prueba, /productos) de forma rapidísima sin tener que programar todo el protocolo HTTP desde cero.



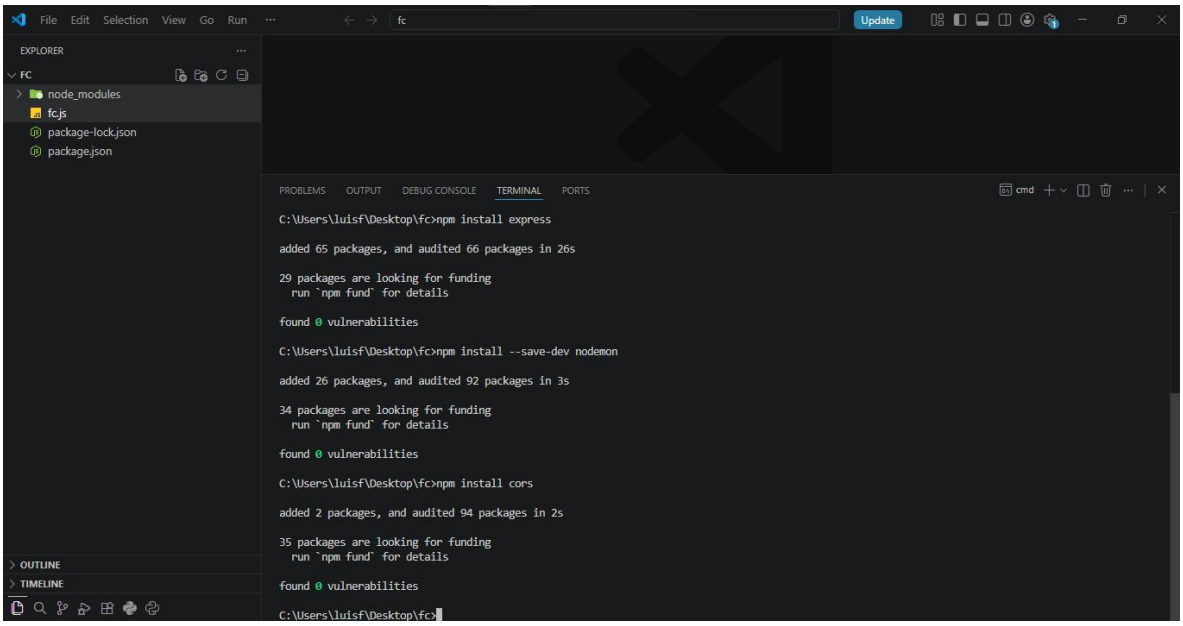
npm install --save-dev nodemon

te reinicia el servidor solito apenas guardas un cambio en el código. Así te ahorrasde estar matando y prendiendo la consola a cada rato con Ctrl + C



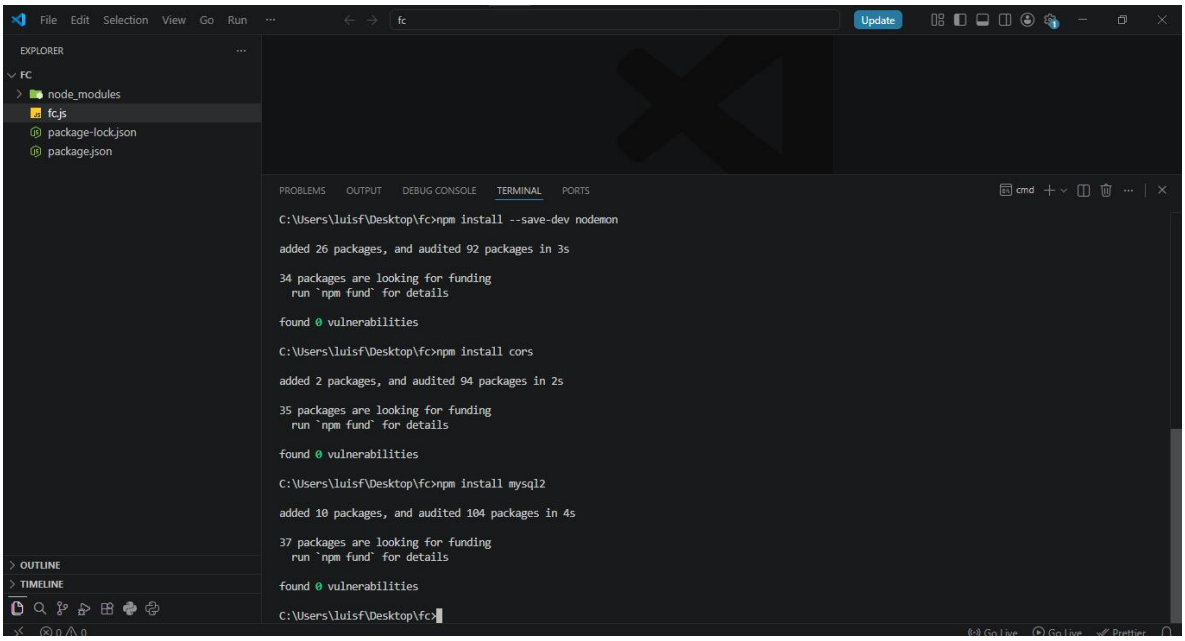
npm install cors

ctúa como un permiso de seguridad para que el navegador no bloquee las peticiones cuando el frontend intente comunicarse con nuestro servidor.



npm install mysql2

Es el puente que le permite a nuestra aplicación hablarle a la base de datos, mandar consultas SQL (SELECT, INSERT) y traer la información.



Fase 1. Investigación y preparación del entorno

1. ¿Qué es Node.js y qué función cumple?

Es básicamente el motor que deja correr código de JavaScript fuera del navegador (en el computador o servidor). Lo usamos para levantar el backend, manejar la lógica y conectar la base de datos sin depender de una página web abierta.

2. ¿Qué es Express y por qué se utiliza para desarrollar una API?

Es una librería (framework) super liviana para Node.js. Nos ahorra un montón de código repetitivo; en vez de armar un servidor desde cero con miles de líneas, con Express montamos rutas (/prueba, /productos) de forma rapidísima y limpia.

3. ¿Qué es una API REST?

Es como el puente o traductor entre diferentes sistemas. Por ejemplo, permite que la interfaz (tu app en React) le pida datos al servidor y este se los devuelva ordenados, casi siempre en un formato llamado JSON que cualquier tecnología entiende fácil.

4. ¿Qué función cumple CORS?

Es un policía de seguridad de los navegadores. Si tu app web y tu servidor están en puertos diferentes (como el 3000 y el 5000), el navegador bloquea la conexión por seguridad. Poner cors() es básicamente decirle al navegador: *"Tranquilo, estos dos sí se conocen, déjalos hablar"*.

5. ¿Qué es MySQL y para qué sirve una base de datos?

MySQL es el programa gestor donde guardamos toda la info de forma segura en tablas con filas y columnas. Sirve para que los datos no se borren cuando apagues la PC, manteniendo todo persistente.

6. ¿Qué información se necesita para establecer una conexión con MySQL?

Básicamente 5 datos fijos:

Host: Dónde está alojada (en este caso nuestra propia PC, o sea 127.0.0.1).

Puerto: Por dónde se comunican (el clásico 3306).

Usuario: Quién entra (normalmente root).

Contraseña: Tu clave personal de acceso.

Database: El nombre exacto de la base de datos (en nuestro caso, inventario)

7. ¿Qué función cumple cada dependencia instalada en el proyecto?

express: Crea el servidor web y maneja las rutas y peticiones.

cors: Evita los bloqueos de seguridad del navegador entre el frontend y el backend.

mysql2: Es el conector para que Node.js pueda hablarle a MySQL y tirarle comandos SQL.

nodemon: El salvavidas de desarrollo que reinicia el servidor automático cada vez que guardas un cambio en el código, para que no tengas que apagar y prender la consola a cada rato.

Fase 2. Diseño de la base de datos (Explicación informal)

1. ¿Por qué elegimos esos tipos de datos para la tabla de productos?

id (INT AUTO_INCREMENT PRIMARY KEY): Usamos un número entero (INT) porque es lo ideal para llevar la cuenta en orden. Con el AUTO_INCREMENT, la base de datos se encarga solita de sumarle 1 a cada producto nuevo que metamos, y al ser la llave principal (PRIMARY KEY), se vuelve el código único de cada artículo.

nombre (VARCHAR(100)): Le ponemos texto variable (VARCHAR) porque nos da campo de sobra (hasta 100 letras) para escribir el nombre del producto sin quedarnos cortos.

categoria (VARCHAR(50)): Igual texto variable, pero un poquito más corto (50 caracteres) para agrupar las cosas (ej. tecnología, ropa, etc.).

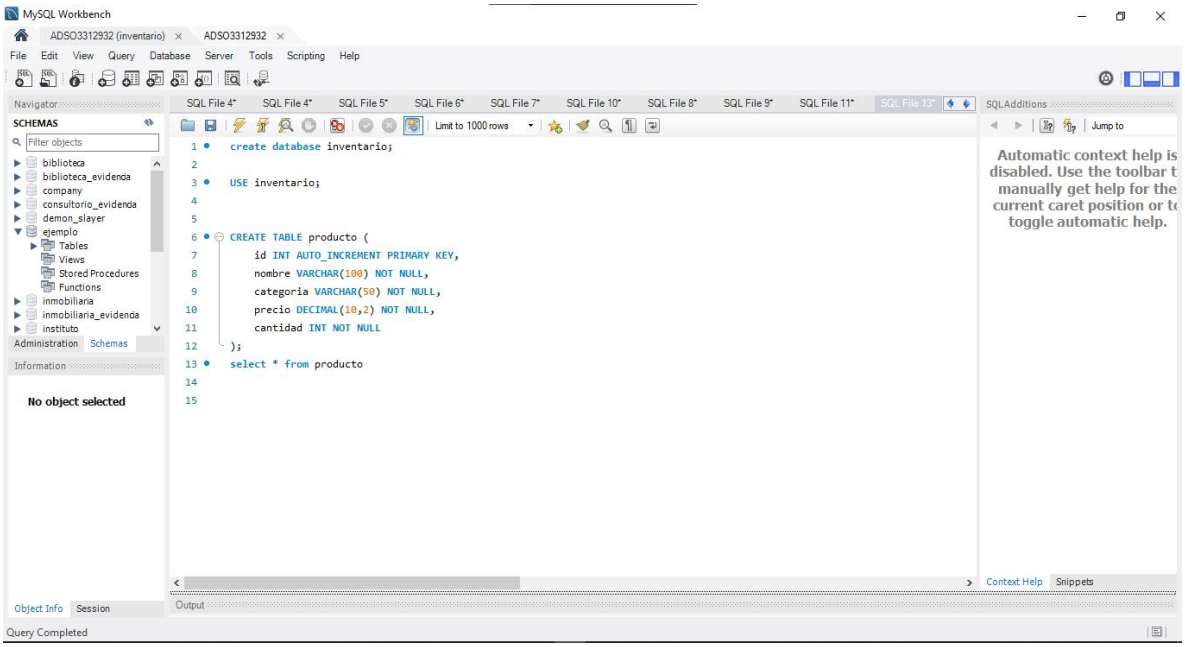
precio (DECIMAL(10,2)): Este es clave. Usamos DECIMAL porque el dinero maneja centavos exactos. Con el (10,2) le decimos que aguante números grandes y que siempre guarde dos decimales fijos (por ejemplo: 1500.00).

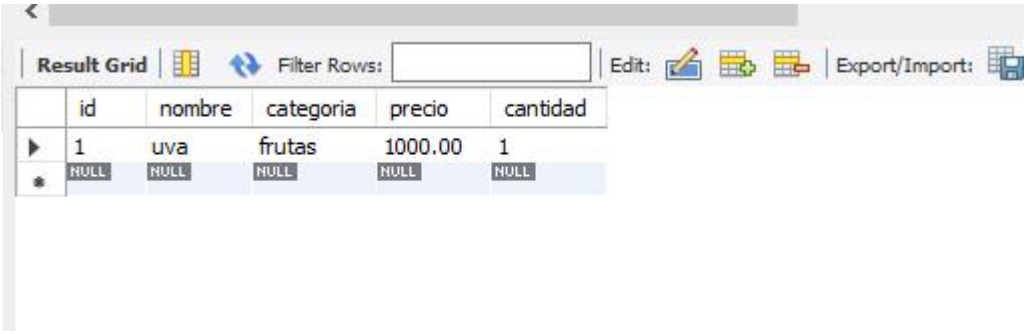
cantidad (INT): Va con números enteros (INT) porque las cosas del inventario se cuentan por unidades completas (no vas a vender media laptop).

2. ¿Por qué el identificador (id) tiene que ser único?

Es obligatorio porque es la única forma que tiene la base de datos para no confundir un producto con otro. Si tenemos dos artículos que se llaman igual, el id es el que nos dice exactamente cuál es cuál, evitando que se armen enredos o se borre el que no es.

3. Evidencia: captura de la estructura de la tabla y de los registros de prueba en MySQL.



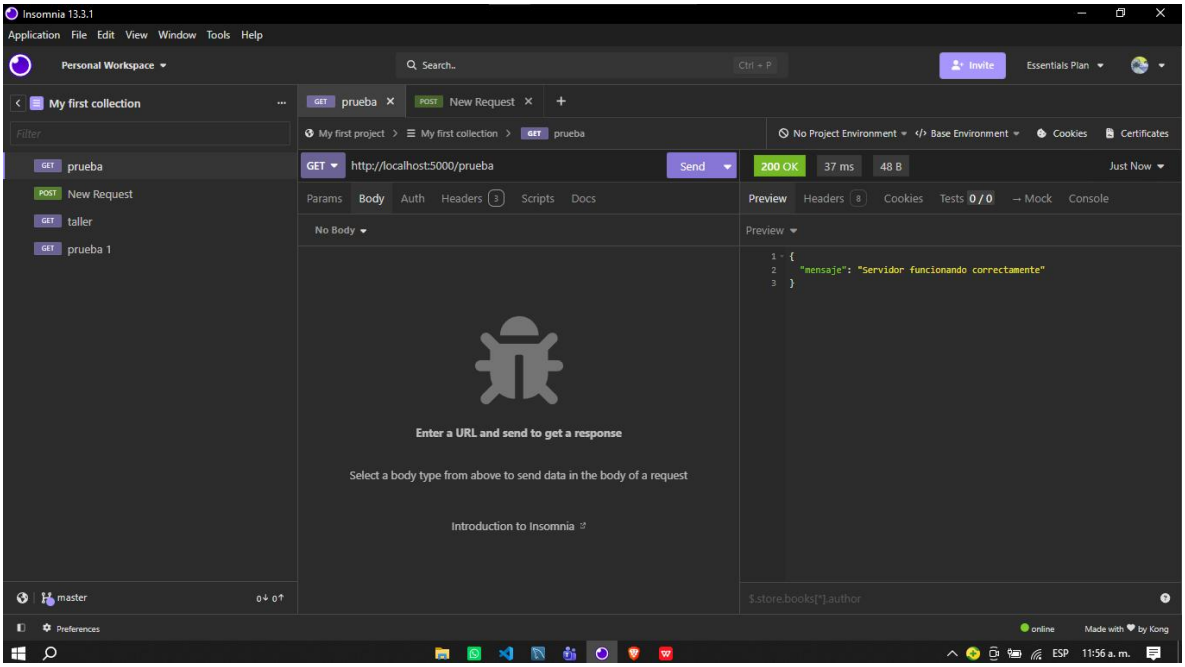


Fase 3. Construcción de la API

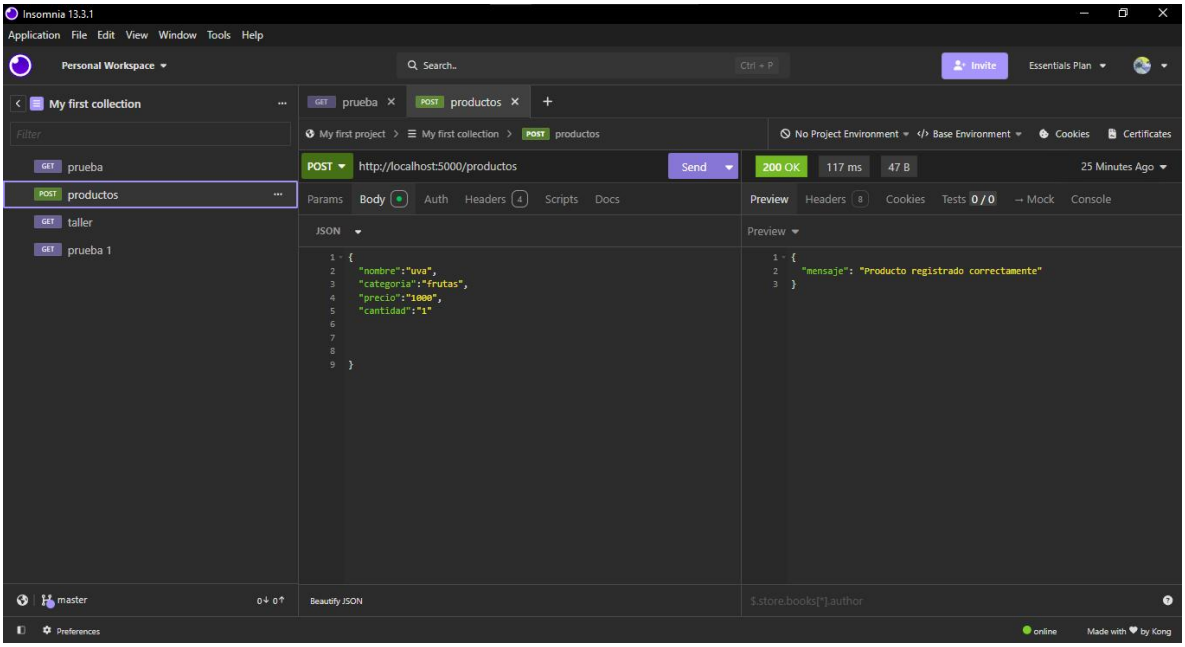
Metodo http	Ruta	Propósito / Función	Resultado Esperado
GET	/prueba	Verificar que el servidor web esté arriba y respondiendo peticiones.	Un JSON con un mensaje de éxito: {"mensaje": "Servidor funcionando correctamente"}.
POST	/productos	Recibir datos en formato JSON desde el cliente para registrar un nuevo producto en la base de datos inventario.	Un JSON confirmando el registro con código HTTP 201: {"mensaje": "Producto registrado correctamente"}.
GET	/productos	Consultar y traer todos los productos que están guardados actualmente en la base de datos.	Un arreglo JSON con la lista completa de todos los artículos almacenados en la tabla producto.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Captura de /prueba:

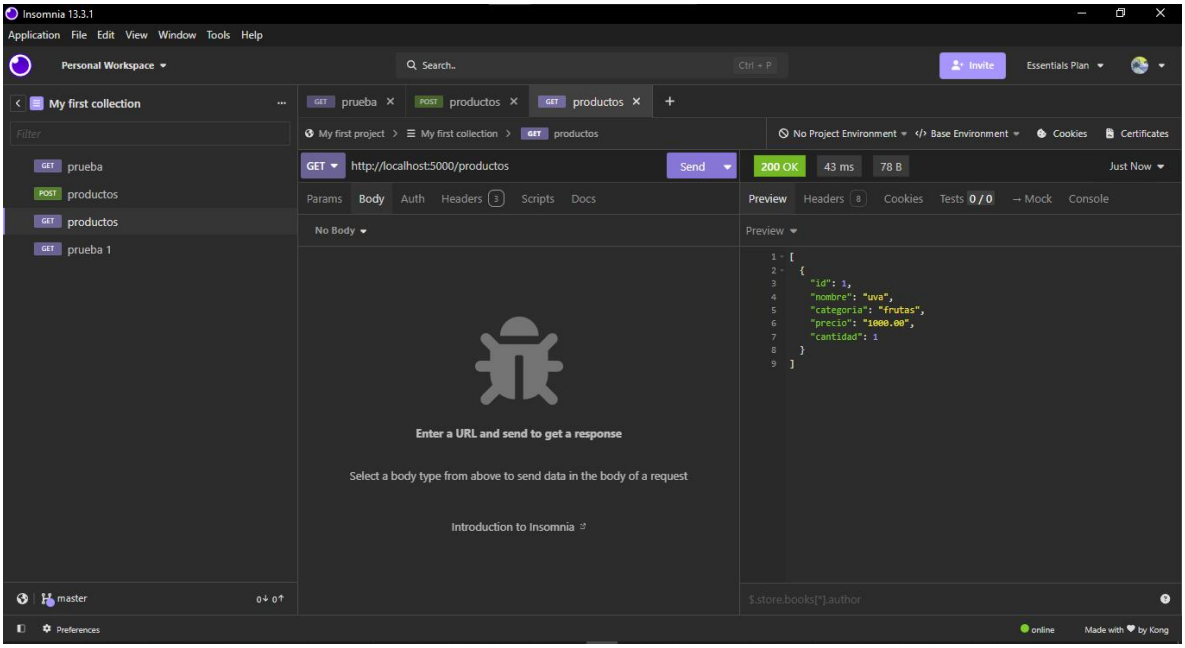


Captura de /productos (POST):



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Captura de /productos (GET):



Fase 4. Revisión y análisis de errores

Configuración del middleware para JSON

¿Cuál es el error? Olvidar poner app.use(express.json()).

¿Por qué ocurre? Porque Express viene pelado y no sabe interpretar los datos en formato JSON que le mandan en el cuerpo (body) de las peticiones.

¿Qué consecuencia tiene? Que el servidor recibe todo vacío (undefined) y los registros nunca se guardan.

¿Cómo debería corregirse? Poner app.use(express.json()) arriba del todo, antes de crear las rutas.

¿Qué prueba permite comprobar la solución? Mandar un POST con datos y ver que el servidor los lea sin bronca.

Conexión con la base de datos

¿Cuál es el error? Tener mal escritos los datos en mysql.createConnection.

¿Por qué ocurre? Poner mal la contraseña, equivocarse de puerto o poner un nombre de base de datos que no existe.

¿Qué consecuencia tiene? La API se queda sorda y ciega, tirando error apenas arranca porque no se pudo enlazar a MySQL.

¿Cómo debería corregirse? Revisar que el host, user, password y database coincidan exactamente con tu gestor.

¿Qué prueba permite comprobar la solución? Ver el mensaje de consola confirmando que la conexión está activa.

Nombres de tablas y campos en las consultas

¿Cuál es el error? Escribir mal el nombre de la tabla o de las columnas en el código SQL.

¿Por qué ocurre? Distracción o cambiarle el nombre en la base de datos y olvidarse de actualizarlo en Node.js.

¿Qué consecuencia tiene? La base de datos rechaza la orden y la API responde con un error de SQL.

¿Cómo debería corregirse? Verificar letra por letra que el string SQL coincida con tu tabla producto.

¿Qué prueba permite comprobar la solución? Hacer un GET o un POST y ver que responda con éxito.

Inserción incorrecta de registros

¿Cuál es el error? Armar las consultas SQL pegando las variables a la brava con símbolos de suma o comillas.

¿Por qué ocurre? Malas prácticas al escribir código rápido.

¿Qué consecuencia tiene? Quedar expuestos a inyecciones SQL y errores de sintaxis cuando mandan textos raros.

¿Cómo debería corregirse? Usar consultas parametrizadas con signos de interrogación ? y un arreglo con los valores.

¿Qué prueba permite comprobar la solución? Meter un producto con apóstrofes o nombres largos sin que la app se caiga.

Respuestas invertidas (éxito y error)

¿Cuál es el error? Mandar un mensaje de "todo bien" cuando la base de datos falló, o viceversa.

¿Por qué ocurre? Enredarse con el bloque if (error) dentro del callback de MySQL.

¿Qué consecuencia tiene? Confundir al usuario o al frontend haciéndole creer que algo se guardó cuando en realidad se perdió.

¿Cómo debería corregirse? Validar bien la variable error para que responda con el mensaje y código adecuado según el caso.

¿Qué prueba permite comprobar la solución? Forzar un error a propósito y revisar que la API cante la falla.

Consulta incorrecta de los registros

¿Cuál es el error? Hacer un SELECT a una tabla que no es o pedir campos inventados.

¿Por qué ocurre? Copiar y pegar código de otro lado sin adaptarlo.

¿Qué consecuencia tiene? Que el GET devuelva datos vacíos o la lista incorrecta.

¿Cómo debería corregirse? Dejar la consulta apuntando bien: SELECT * FROM producto.

¿Qué prueba permite comprobar la solución? Ejecutar el GET en Insomnia y ver que traiga los productos reales.

Validación de los datos recibidos

¿Cuál es el error? Confiar en que el cliente siempre va a mandar la información completa y bien escrita.

¿Por qué ocurre? No poner filtros previos en el código antes de mandar los datos a MySQL.

¿Qué consecuencia tiene? Se guardan registros incompletos, vacíos o con datos absurdos.

¿Cómo debería corregirse? Poner un if que revise si faltan campos obligatorios antes de hacer el INSERT.

¿Qué prueba permite comprobar la solución? Mandar un POST sin precio y ver que la API lo rechace de una.

Uso adecuado de códigos de estado HTTP

¿Cuál es el error? Devolver siempre código 200 OK para absolutamente todo, incluso cuando hay errores graves.

¿Por qué ocurre? No configurar los métodos res.status().

¿Qué consecuencia tiene? El cliente piensa que todo salió perfecto cuando en realidad hubo un fallo.

¿Cómo debería corregirse? Usar 200 para OK, 201 para creado, 400 para datos malos y 500 para errores del servidor.

¿Qué prueba permite comprobar la solución? Ver los colores de los códigos en Insomnia (verde para éxito, naranja/rojo para errores).

Manejo de errores en las consultas

¿Cuál es el error? Dejar el servidor mudo o sin respuesta cuando la consulta SQL falla.

¿Por qué ocurre? No atrapar el error dentro de los bloques de respuesta de la base de datos.

¿Qué consecuencia tiene? Que la API se cuelgue o se quede "pensando" eternamente sin responderle nada al cliente.

¿Cómo debería corregirse? Poner siempre un return res.status(500).json(...) dentro de la validación del error.

¿Qué prueba permite comprobar la solución? Dañar a propósito la consulta y verificar que la API devuelva un JSON de error controlado.

Protección de las credenciales de conexión

¿Cuál es el error? Dejar la contraseña de la base de datos escrita a la vista en el código fuente.

¿Por qué ocurre? Por pereza de configurar variables de entorno al principio.

¿Qué consecuencia tiene? Que si subes el código a GitHub, cualquiera puede robarte el acceso a tu base de datos.

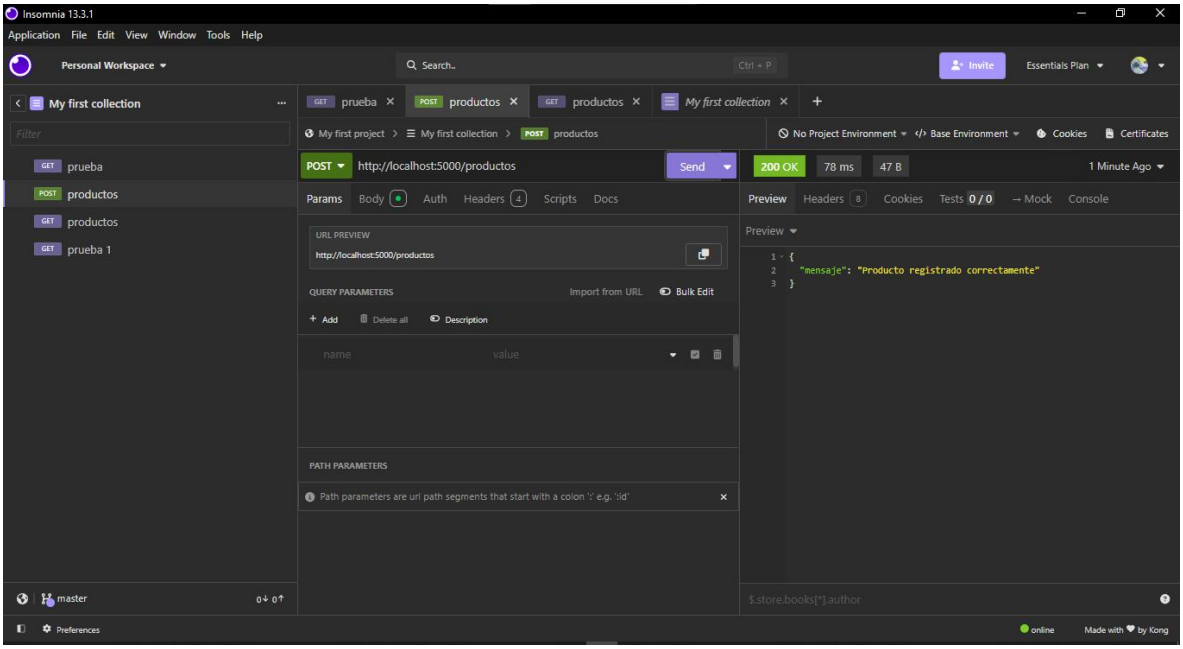
¿Cómo debería corregirse? Usar el paquete dotenv para sacar las credenciales a un archivo oculto .env.

¿Qué prueba permite comprobar la solución? Revisar que el código principal no tenga claves escritas antes de subirlo.

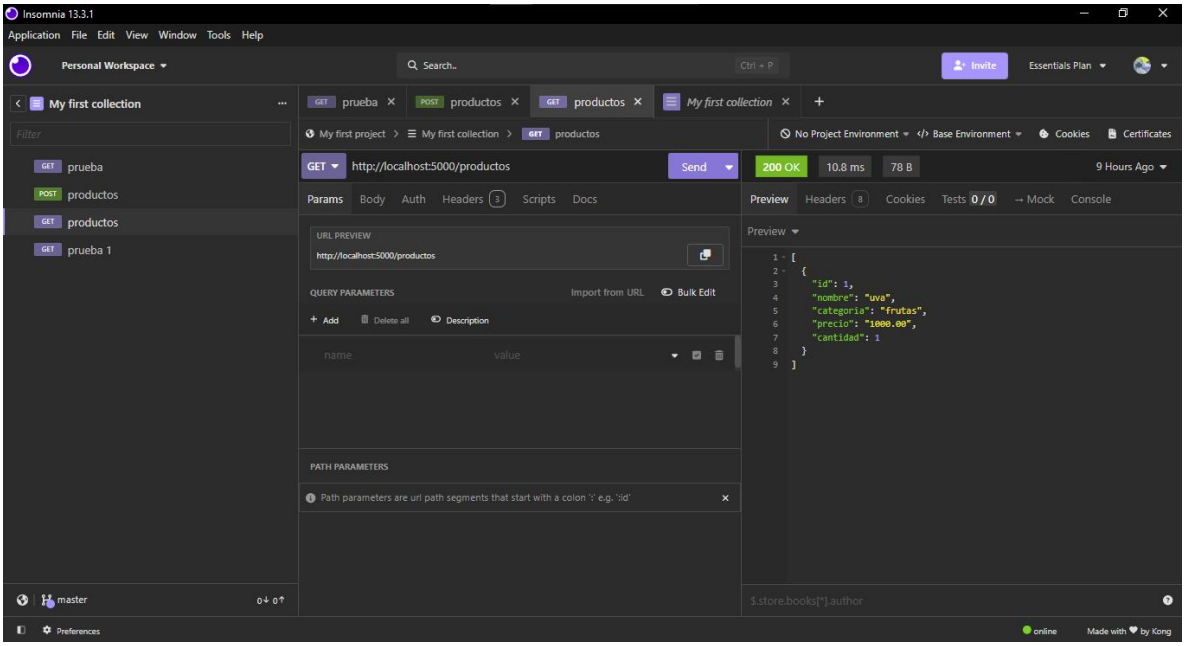
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Fase 5. Pruebas funcionales

- Registro exitoso de un producto.

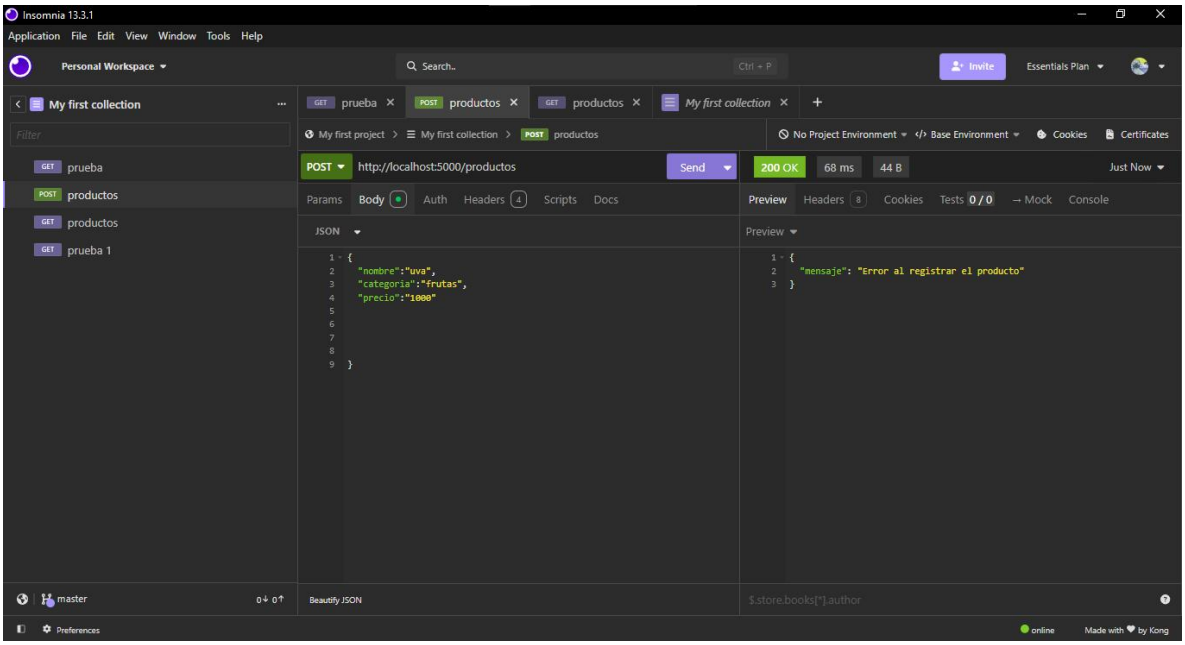


- Consulta de productos registrados.

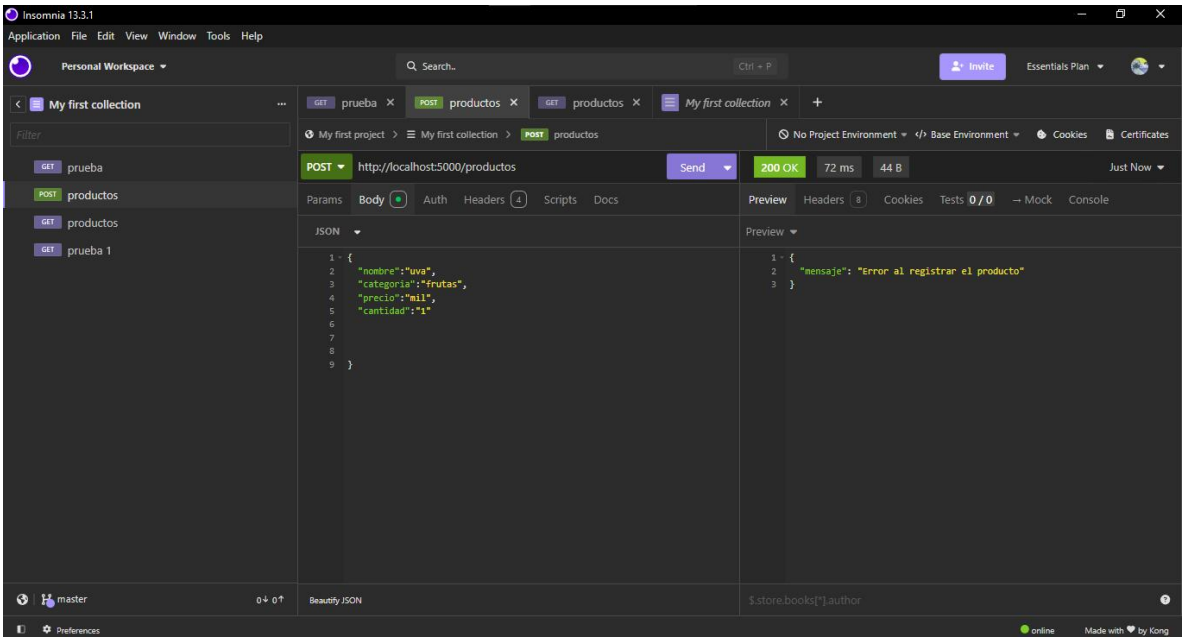


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

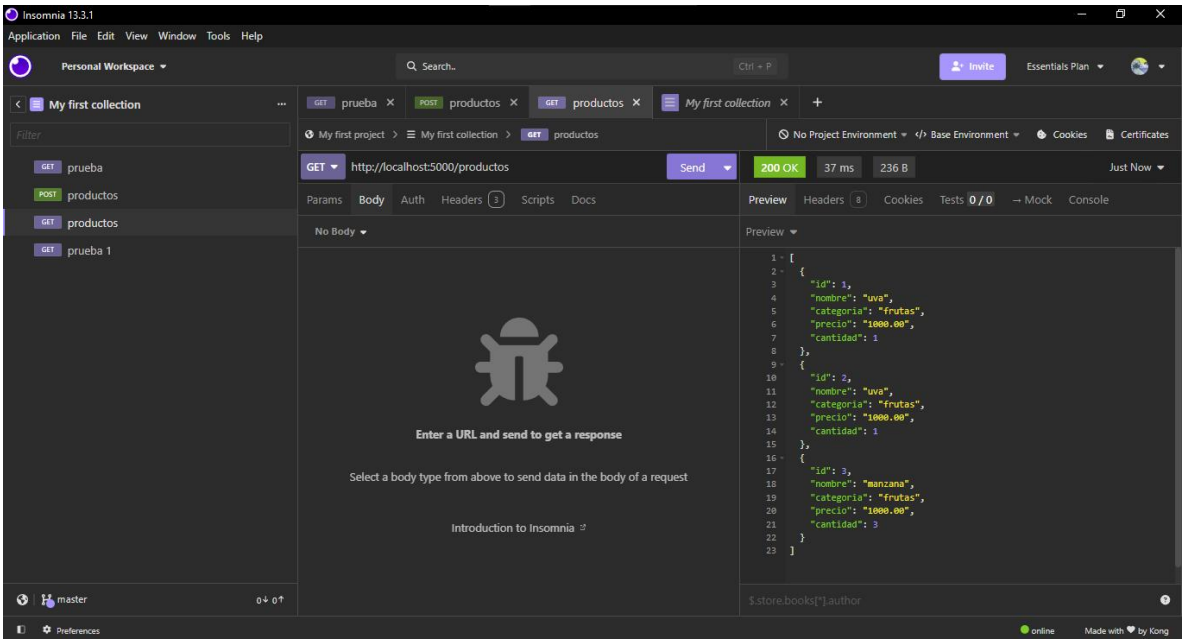
- Envío de información incompleta.



- Envío de valores inválidos para el precio o la cantidad.



- Respuesta del servidor cuando ocurre un error.



***Preguntas de análisis**

· **¿Qué diferencia existe entre GET y POST?**

El GET se usa solo para consultar información y los datos viajan a la vista en la URL. El POST se usa para enviar y registrar datos nuevos, los cuales viajan seguros y ocultos en el cuerpo (body) de la petición.

· **¿Por qué se debe validar la información antes de almacenarla?**·

Para evitar que se guarden datos vacíos, incompletos o maliciosos que dañen la base de datos o la lógica del programa.

¿Qué sucede cuando la API intenta insertar un registro en una tabla que no existe?

La base de datos rechaza la orden de una y le manda un error al servidor, el cual debemos atrapar para responderle al cliente con un fallo controlado (500) en lugar de dejar que la app se caiga.

· **¿Por qué es importante distinguir entre una respuesta exitosa y una respuesta de error?**

Para que la aplicación o el usuario sepan exactamente si la acción se completó o si hubo un fallo y así mostrar el mensaje que es en pantalla.

¿Qué diferencia existe entre los errores de conexión y los errores de consulta SQL?

El error de conexión pasa cuando la API ni siquiera puede hablar con la base de datos (por ejemplo, clave mala o servidor apagado). El error de consulta pasa cuando sí está conectada, pero la orden SQL está mal escrita.

· **¿Por qué se deben utilizar consultas parametrizadas?**

Porque separan el código SQL de los datos que escribe el usuario, bloqueando los ataques de inyección SQL y evitando errores por comillas mal puestas.

¿Qué función cumple CORS cuando la API se consume desde Angular?

· Funciona como un permiso de seguridad del navegador para dejar que una interfaz que corre en otro puerto pueda comunicarse libremente con el servidor sin bloqueos.

· **¿Qué ventajas ofrece almacenar la información en una base de datos en línea**

Que los datos quedan centralizados, seguros y disponibles a cualquier hora desde cualquier lugar, con respaldos automáticos

·**¿Por qué es importante probar los casos incorrectos y no solamente los exitosos?**

Para asegurarnos de que el sistema sea resistente, sepa reaccionar ante imprevistos y no le muestre fallos raros o se quede congelado para el usuario final.

¿Cómo se puede proteger la información de conexión a la base de datos?

Usando variables de entorno con paquetes como dotenv y evitando subir contraseñas o datos sensibles a repositorios públicos como GitHub.